# STEMMate A4 Individual Evidence

Name: Johnpaul Nkhwazi

Student number: 225175576  Practical group: 3

Role: 7 Tester  Date: 6 November 2026

**Review draft. Complete the missing details and record your own verification before submitting.**

## Contribution and learning statement

563 words. Supporting evidence below is excluded from the statement word count.

My assigned responsibility for STEMMate Namibia Assessment 4 is Role 7, tester. My contribution package focuses on the offline session planning workflow and the evidence needed to assess its reliability.

The testing work uses the actual classes in the repository rather than treating the earlier HTML prototype as the final implementation. Java tests were added under client/src/test/java, with further server and standalone prototype tests in their own folders. A small runner compiles the relevant production classes and executes each case in a separate Java process. Each process uses an isolated data folder, so the tests do not alter ordinary application data. The package contains the source, individual logs, machine-readable results, JUnit-format XML and source hashes.

Forty automated cases were executed: twenty-five client cases, eleven server cases and four standalone prototype cases. Thirty passed and ten failed. Coverage includes activity filtering, offline activity selection, plan creation, required elements, timing validation, autosaved drafts, idempotent local saves, reconnect handling, retry, partial confirmation, batch sizes, HTTP payloads, server validation and storage errors. Two restart cases use a writer process that exits and a different reader process that verifies stored data. Those checks passed for the desktop plan, draft and outbox files and for the prototype text outbox. They demonstrate Java file persistence, not server SQLite persistence.

## Contribution and learning statement continued

An important finding is that persistent data can exist while recovery is still incomplete. The desktop outbox retains a record marked SYNCING, but its pending query only selects WAITING_TO_SYNC and FAILED_RETRY. After an interrupted attempt, that record can therefore remain stored without becoming eligible for another upload. A second failure shows that a plan write can fail after its outbox entry has already been written. These findings support a proposed recovery decision: preserve the data and restore interrupted work to a safe, retryable state. Idempotency must prevent a repeated attempt from creating a second plan.

The evidence also distinguishes the repository's two implementation paths. The main desktop client uses a JSON interface intended for the Ballerina service. The separate standalone prototype uses the line-format interface provided by the Java server. Their routes, identifiers and timestamp formats cannot simply be mixed. The test report records this as a pairing and integration issue. It also records malformed JSON output, incorrect confirmation parsing, missing server-side FR4 checks, weak safety-content validation, missing label associations and an outbox write failure that is not returned to its caller.

My main lesson from this work is to separate an observed result from a claim that the whole system is finished. Source inspection is useful for finding missing label bindings, but it is not a keyboard or screen-reader session. A transport double can test retry and response handling, but it does not prove live server synchronisation. The remaining GUI, network and SQLite checks are listed honestly as unexecuted. My next step is to review the files, reproduce the runner where possible, share the defects with the developers and ensure the representative uses the actual FR and QR traceability. I must add real peer evidence and record any further verification with its actual date.

## Evidence map decision and requirement trace

All new artefacts below are dated 6 November 2026. They are AI-assisted work prepared for this contribution; no earlier personal repository history is claimed.

| Date | Artefact | Evidence |
|---|---|---|
| 6 Nov 2026 | client/src/test/java/org/stemmate/testing/Role7ClientTests.java | 25 executed client cases. |
| 6 Nov 2026 | server/src/test/java/stemmate/server/Role7ServerTests.java | 11 server validation and route cases. |
| 6 Nov 2026 | prototype/tests/Role7PrototypeTests.java | Four codec/outbox cases. |
| 6 Nov 2026 | scripts/RunRole7Tests.java | Repeatable compilation and execution command. |
| 6 Nov 2026 | docs/tests/role7/evidence/client/J09.txt | Real writer-process exit and reader-process verification. |
| 6 Nov 2026 | docs/tests/role7/DEFECT_LOG.md | Ten honest issue records with reproduction references. |
| 6 Nov 2026 | docs/tests/role7/TRACEABILITY_MATRIX.md | Requirement, component, case, result and issue links. |

### Decision to review and defend

Recover an interrupted SYNCING record to a retryable state while retaining the stored plan. J24 demonstrates that the record currently remains stored but is not returned by findPending. Keeping the record permanently SYNCING was rejected because it can never complete through the normal retry path. Deleting it was rejected because unconfirmed work could be lost. A retry needs the existing plan/item identity so that server idempotency prevents a duplicate.

This is a testing recommendation for my review and explanation. It is not a claim that I implemented a recovery fix or that the group has adopted it.

### Requirement through contribution to result

FR5 and QR1 -> LocalOutboxRepository -> J24 interrupted-state test -> stored SYNCING with retry queue size zero -> FAIL -> D06 -> proposed startup recovery and retest. A second trace is FR6 -> LocalSessionPlanStore -> J09 -> complete draft and plan restored in a separate reader JVM -> PASS.

## Reflection and peer contribution evidence

### Failure or mistaken assumption

At the start I did not understand the full evidence required from the tester. The completed test records show why saving a file or displaying a status is not enough to prove safe recovery. The normal restart checks passed, while the interrupted-state and partial-write checks failed. My response is to preserve both kinds of result and identify the exact remaining work. I still need to review or reproduce the generated material and coordinate the final GUI and live-server checks with the developers.

### Peer contribution ratings

Use the team rating scale, or clearly state a 1-5 scale if the group has none. Add the real name, rating and dated evidence for each member. No ratings have been invented.

| Role | Member and rating | Dated supporting evidence |
|---|---|---|
| 1 PM / representative | To complete | To complete |
| 2 Requirements | To complete | To complete |
| 3 Architecture | To complete | To complete |
| 4 Data / resilience | To complete | To complete |
| 5 Backend development | To complete | To complete |
| 6 Development / UX | To complete | To complete |
| 8 Evaluation / handover | To complete | To complete |